

TP Chapitre 1 : Représentation Numérique

Travaux Pratiques

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Université Cheikh Anta Diop de Dakar

Environnement de travail

Ce TP doit être réalisé dans l'environnement Docker du cours.

```
# Linux/macOS
docker run -it -v $(pwd):/workspace elbachirtoure/c-advanced-course:v1.0

# Windows (Git Bash)
docker run -it -v /${PWD}:/workspace elbachirtoure/c-advanced-course:v1.0

cd /workspace
```

1 Exercice 1 : Affichage Binaire et Hexadécimal (30 min)

Objectif

Implémenter des fonctions d'affichage binaire et hexadécimal pour visualiser la représentation mémoire des entiers. Concepts : conversions de bases, MSB/LSB, format hex 2 caractères par octet.

1.1 Travail demandé

Créer un fichier `affichage.c` avec les fonctions suivantes :

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 // Affiche les n bits d'une valeur (MSB à gauche)
5 // Exemple: print_bits(0x0B, 8) affiche "00001011"
6 void print_bits(uint64_t value, int n_bits) {
7     // À implémenter
8 }
9
10 // Affiche size octets en hexadécimal (2 chars par octet)
11 // Exemple: pour int x = 0x12345678, affiche "78 56 34 12" (little-endian)
12 void print_hex(void *ptr, size_t size) {
13     // À implémenter
14 }
15
16 int main(void) {
17     uint8_t u8 = 0xB5;
18     int8_t s8 = -5;
19     uint32_t u32 = 0x12345678;
20
21     printf("u8 = 0x%02X\n", u8);
22     printf("  Binaire: "); print_bits(u8, 8); printf("\n");
23     printf("  Hex mem: "); print_hex(&u8, sizeof(u8)); printf("\n\n");
24
25     printf("s8 = %d\n", s8);
26     printf("  Binaire: "); print_bits(*(uint8_t*)&s8, 8); printf("\n");
27     printf("  Hex mem: "); print_hex(&s8, sizeof(s8)); printf("\n\n");
28
29     printf("u32 = 0x%08X\n", u32);
30     printf("  Binaire: "); print_bits(u32, 32); printf("\n");
31     printf("  Hex mem: "); print_hex(&u32, sizeof(u32)); printf("\n\n");
32
33     return 0;
34 }
```

Questions :

1. Pourquoi l'affichage hex en mémoire de `u32` est-il "inversé" par rapport à la valeur ?
2. Quel est le motif binaire de `s8 = -5` ? Vérifiez avec la formule $-x = \sim x + 1$.

2 Exercice 2 : Signed vs Unsigned et Extension de Signe (45 min)

Objectif

Explorer les conversions entre types signed/unsigned et observer l'extension de signe. Concepts : B2U, B2T, zero-extension, sign-extension, pièges des comparaisons.

2.1 Partie A : Conversions et extensions

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 int main(void) {
5     int8_t a = -1;
6
7     // Conversions
8     uint8_t b = a;          // Même motif, interprétation différente
9     int16_t c = a;          // Extension de signe
10    uint16_t d = a;          // Extension puis unsigned
11    uint16_t e = (uint8_t)a; // Cast puis extension
12
13    printf("a (int8) = %4d | hex: ", a);
14    print_hex(&a, sizeof(a)); printf("\n");
15
16    printf("b (uint8) = %4u | hex: ", b);
17    print_hex(&b, sizeof(b)); printf("\n");
18
19    printf("c (int16) = %4d | hex: ", c);
20    print_hex(&c, sizeof(c)); printf("\n");
21
22    printf("d (uint16) = %5u | hex: ", d);
23    print_hex(&d, sizeof(d)); printf("\n");
24
25    printf("e (uint16) = %5u | hex: ", e);
26    print_hex(&e, sizeof(e)); printf("\n");
27
28    return 0;
29 }
```

Questions :

1. Expliquez la différence entre d et e.
2. Comment le bit de signe se propage-t-il dans c ?

2.2 Partie B : Pièges des comparaisons

```
1 #include <stdio.h>
2
3 int main(void) {
4     int i = -1;
5     unsigned int u = 0;
6
7     printf("i = %d, u = %u\n", i, u);
8     printf("i < u ? %s\n", (i < u) ? "vrai" : "faux");
9
10    // Prédire le résultat avant d'exécuter
11    printf("-1 < 0U ? %s\n", (-1 < 0U) ? "vrai" : "faux");
12    printf("2147483647 > -2147483648 ? %s\n",
13           (2147483647 > -2147483648) ? "vrai" : "faux");
14    printf("2147483647U > -2147483648 ? %s\n",
15           (2147483647U > -2147483648) ? "vrai" : "faux");
16
17    return 0;
18 }
```

Compilez avec gcc -Wall et observez les warnings.

3 Exercice 3 : Overflow et Détection (45 min)

Objectif

Observer le comportement aux limites et implémenter une détection d'overflow sûre. Concepts : wrap-around unsigned, UB signed, détection avant opération.

3.1 Partie A : Observation des overflows

```
1 #include <stdio.h>
2 #include <stdint.h>
3 #include <limits.h>
4
5 int main(void) {
6     // Unsigned : comportement défini (wrap-around)
7     uint8_t u = 255;
8     printf("Avant: u = %u\n", u);
9     u = u + 1;
10    printf("Après u+1: u = %u\n\n", u);
11
12    u = 0;
13    printf("Avant: u = %u\n", u);
14    u = u - 1;
15    printf("Après u-1: u = %u\n\n", u);
16
17    // Signed : UB mais observons ce qui se passe
18    int8_t s = 127;
19    printf("Avant: s = %d\n", s);
20    s = s + 1; // UB!
21    printf("Après s+1: s = %d (UB!)\n", s);
22
23    return 0;
24 }
```

Compilez avec `gcc -O0` puis `gcc -O2`. Le résultat est-il le même ?

3.2 Partie B : Détection sûre d'overflow

Implémentez les fonctions suivantes :

```
1 #include <stdbool.h>
2 #include <limits.h>
3
4 // Retourne true si a + b causerait un overflow (unsigned)
5 bool will_overflow_add_u32(uint32_t a, uint32_t b) {
6     // À implémenter
7 }
8
9 // Retourne true si a + b causerait un overflow (signed)
10 bool will_overflow_add_i32(int32_t a, int32_t b) {
11     // À implémenter : tester AVANT l'opération
12 }
13
14 // Test
15 int main(void) {
16     printf("UINT32_MAX + 1 overflow? %s\n",
17           will_overflow_add_u32(UINT32_MAX, 1) ? "oui" : "non");
18     printf("INT32_MAX + 1 overflow? %s\n",
19           will_overflow_add_i32(INT32_MAX, 1) ? "oui" : "non");
20     printf("100 + 200 overflow? %s\n",
21           will_overflow_add_i32(100, 200) ? "oui" : "non");
22     return 0;
23 }
```

4 Exercice 4 : IEEE 754 et Précision (45 min)

Objectif

Explorer la représentation IEEE 754 et les erreurs de précision. Concepts : décomposition float, valeurs spéciales, non-associativité.

4.1 Partie A : Décomposition d'un float

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 typedef union {
5     float f;
6     uint32_t u;
7     struct {
8         uint32_t mantissa : 23;
9         uint32_t exponent : 8;
10        uint32_t sign : 1;
11    } parts;
12 } float_bits;
13
14 void analyze_float(float value) {
15     float_bits fb;
16     fb.f = value;
```

```

17     printf("Valeur:    %f\n", value);
18     printf("Hex:      0x%08X\n", fb.u);
19     printf("Signe:    %u\n", fb.parts.sign);
20     printf("Exposant: %u (biaisé), %d (réel)\n",
21           fb.parts.exponent, (int)fb.parts.exponent - 127);
22     printf("Mantisse: 0x%06X\n", fb.parts.mantissa);
23     printf("\n");
24 }
25
26
27 int main(void) {
28     analyze_float(1.0f);
29     analyze_float(-1.0f);
30     analyze_float(13.0f);
31     analyze_float(0.5f);
32     analyze_float(0.1f); // Observer la mantisse!
33
34     return 0;
35 }

```

4.2 Partie B : Valeurs spéciales et précision

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int main(void) {
5      // Valeurs spéciales
6      float inf = 1.0f / 0.0f;
7      float ninf = -1.0f / 0.0f;
8      float nan_val = 0.0f / 0.0f;
9
10     printf("+inf: "); analyze_float(inf);
11     printf("NaN: "); analyze_float(nan_val);
12
13     printf("nan == nan ? %s\n", (nan_val == nan_val) ? "vrai" : "faux");
14     printf("+0 == -0 ? %s\n", (0.0f == -0.0f) ? "vrai" : "faux");
15
16     // Non-associativité
17     float a = (3.14f + 1e10f) - 1e10f;
18     float b = 3.14f + (1e10f - 1e10f);
19     printf("\n(3.14 + 1e10) - 1e10 = %f\n", a);
20     printf("3.14 + (1e10 - 1e10) = %f\n", b);
21     printf("a == b ? %s\n", (a == b) ? "vrai" : "faux");
22
23     // Comparaison correcte
24     float x = 0.1f + 0.2f;
25     float y = 0.3f;
26     printf("\n0.1 + 0.2 == 0.3 ? %s\n", (x == y) ? "vrai" : "faux");
27     printf("Avec epsilon: %s\n", (fabsf(x - y) < 1e-6f) ? "vrai" : "faux");
28
29     return 0;
30 }

```

5 Exercice 5 : Manipulation de Bits (30 min)

Objectif

Pratiquer les opérations bit à bit et les shifts. Concepts : AND, OR, XOR, NOT, shifts logiques/arithmétiques, extraction de champs.

```

1  #include <stdio.h>
2  #include <stdint.h>
3
4  int main(void) {
5      uint8_t x = 0xB5; // 10110101
6
7      printf("x = 0x%02X = ", x); print_bits(x, 8); printf("\n\n");
8
9      // Opérations de base
10     printf("x & 0x0F = 0x%02X = ", x & 0x0F);
11     print_bits(x & 0x0F, 8); printf("\n");
12
13     printf("x | 0x0F = 0x%02X = ", x | 0x0F);
14     print_bits(x | 0x0F, 8); printf("\n");
15
16     printf("x ^ 0xFF = 0x%02X = ", x ^ 0xFF);
17     print_bits(x ^ 0xFF, 8); printf("\n");
18
19     printf("x << 2 = 0x%02X = ", (uint8_t)(x << 2));
20     print_bits((uint8_t)(x << 2), 8); printf("\n");
21
22     printf("x >> 3 = 0x%02X = ", x >> 3);
23     print_bits(x >> 3, 8); printf("\n\n");
24 }

```

```

25 // Manipulation de bits individuels
26 printf("Bit 5 de x: %d\n", (x >> 5) & 1);
27
28 uint8_t y = x | (1 << 3); // Mettre bit 3 à 1
29 printf("Après set bit 3: 0x%02X\n", y);
30
31 y = x & ~(1 << 7); // Mettre bit 7 à 0
32 printf("Après clear bit 7: 0x%02X\n", y);
33
34 y = x ^ 1; // Toggle bit 0
35 printf("Après toggle bit 0: 0x%02X\n", y);
36
37 // Shifts arithmétiques vs logiques
38 uint32_t u = 0x80000000;
39 int32_t s = 0x80000000;
40
41 printf("u = 0x%08X (unsigned: %u)\n", u, u);
42 printf("s = 0x%08X (signed: %d)\n", s, s);
43 printf("u >> 4 = 0x%08X\n", u >> 4);
44 printf("s >> 4 = 0x%08X\n", s >> 4);
45
46 return 0;
47 }

```

Livrables

- Fichiers sources : exo1.c, exo2.c, exo3.c, exo4.c, exo5.c
- Un fichier reponses.txt avec les réponses aux questions
- Compiler avec : gcc -Wall -Wextra -std=c11